

NIT2112

Object Oriented Programming

Session 06

Exception Handling and Debugging

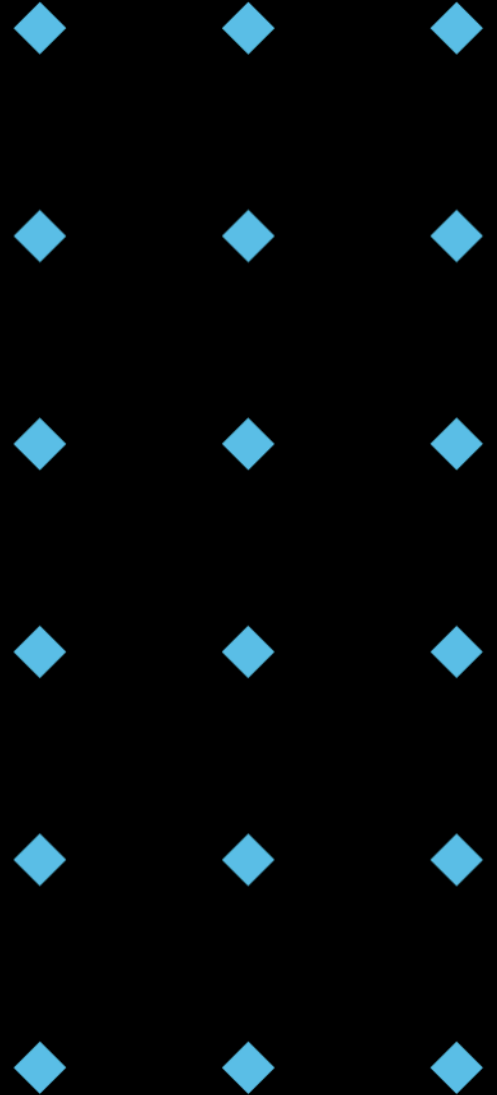
Acknowledgement of Country



Victoria University acknowledges, recognises and respects the Ancestors, Elders and families of the Bunurong/Boonwurrung, Wadawurrung and Wurundjeri/Woiwurrung of the Kulin who are the traditional owners of University land in Victoria, the Gadigal and Guring-gai of the Eora Nation who are the traditional owners of University land in Sydney, and the Yugara/YUgarapul people and Turrbal people living in Meanjin (Brisbane).

Session Outline

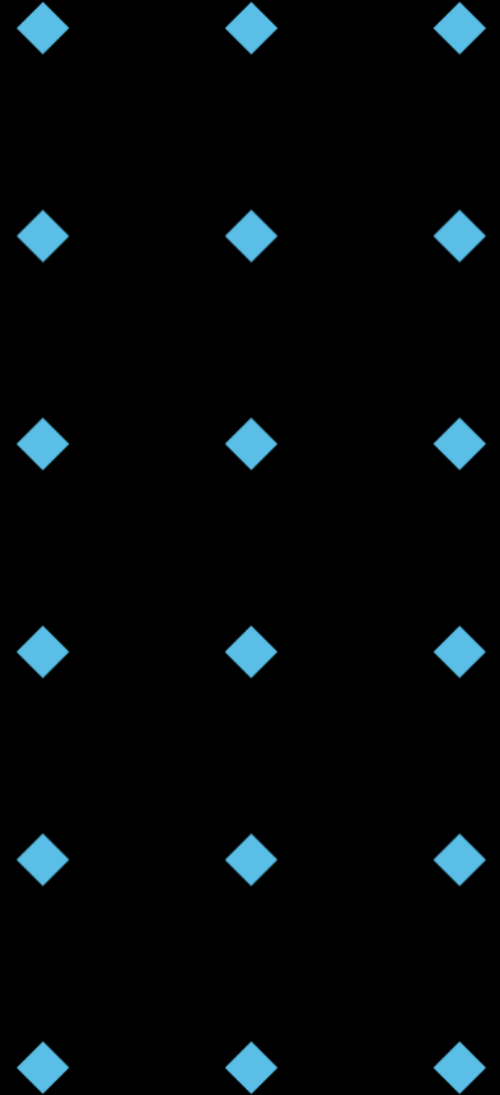
- ◆ Exception Handling Fundamentals (What & Why)
- ◆ Using *try...except...else...finally*
- ◆ Exception Chaining & Propagation
- ◆ Best Practices for Robust Error Handling



The Importance of Handling the Unexpected

Why it Matters:

- ◆ **Robustness:** Prevents programs from crashing unexpectedly. A crash is often a poor user experience.
- ◆ **User Experience:** Allows providing helpful error messages instead of cryptic tracebacks.
- ◆ **Reliability:** Ensures critical operations (like saving data or releasing resources) can complete even if errors occur.
- ◆ **Debugging:** Well-handled exceptions make it easier to find and fix the root cause of problems.
- ◆ **Control Flow:** Provides a structured way to deal with unusual or error conditions separate from the main logic.



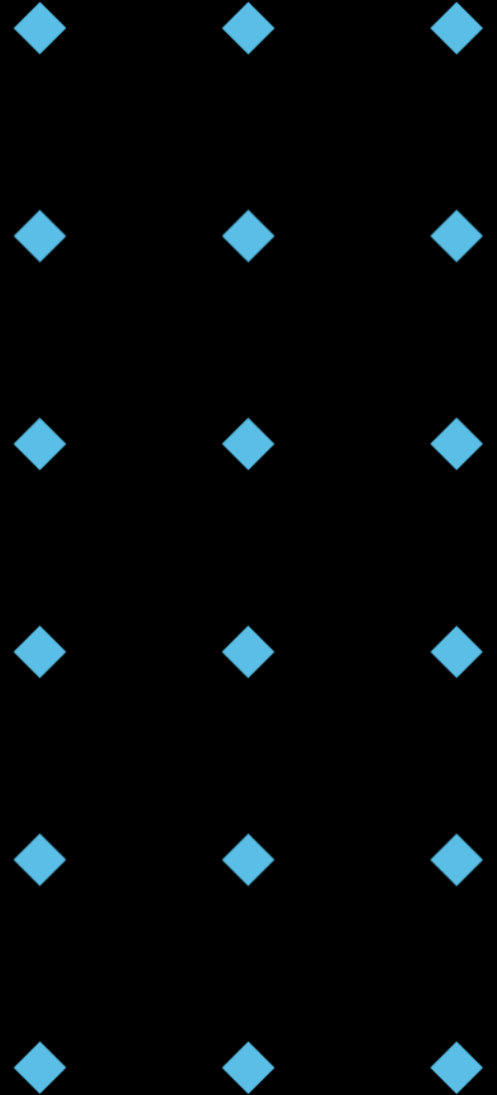
Not All Problems Are Created Equal

Syntax Errors:

- ◆ Errors in the **structure** or **grammar** of the code.
- ◆ Detected by the Python interpreter **before** execution begins.
- ◆ Must be fixed for the program to run at all.
- ◆ Examples: *SyntaxError* (e.g., missing colon, incorrect indentation), *IndentationError*.

Exceptions:

- ◆ Errors that occur during the **execution (runtime)** of a syntactically correct program.
- ◆ **Disrupt** the normal flow of instructions.
- ◆ Can often be anticipated and handled programmatically.
- ◆ Examples: *ZeroDivisionError*, *FileNotFoundError*, *TypeError*.



What Happens When Things Go Wrong?

When an operation fails at runtime:

- An **Exception Object** is created. This object contains information about the error (type, message, location).
- The normal program flow stops.
- Python looks for an Exception Handler (an `except` block) that matches the type of exception raised.
- If a matching handler is found, the code in that handler is executed. The program then continues after the `try...except` block (or jumps to `finally` if present).
- If no handler is found, the program terminates and prints a Traceback.

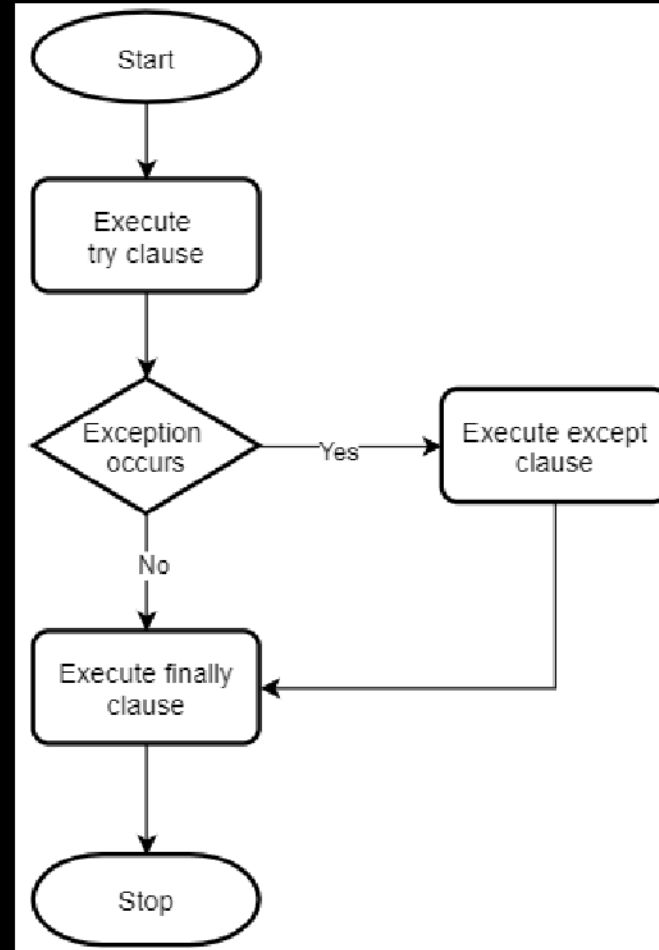


Image from Python Tutorial, 2021, <https://www.pythontutorial.net/python-basics/python-try-except-finally/>

The Basics: Catching and Handling Exceptions

Common Built-in Exceptions

- ♦ *ZeroDivisionError*: Attempting to divide by zero.

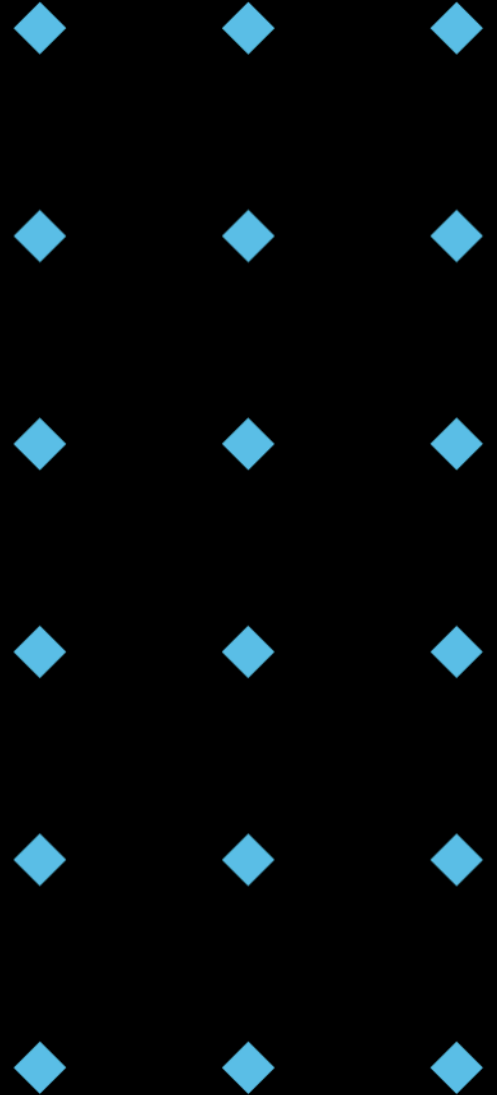
```
1 numerator = 10
2 denominator = 0
3 result = numerator / denominator # Raises ZeroDivisionError
```

- ♦ *TypeError*: Applying an operation or function to an object of an inappropriate type.

```
1 number = 5
2 text = "apples"
3 result = number + text # Raises TypeError (int + str)
```

- ♦ *IndexError*: Trying to access a sequence (list, tuple, string) element using an index that is out of range.

```
1 my_list = [10, 20, 30]
2 item = my_list[3] # Raises IndexError (valid indices are 0, 1, 2)
```



Common Built-in Exceptions

- ♦ *KeyError*: Trying to access a dictionary key that does not exist.

```
1 student_grades = {"Alice": 90, "Bob": 85}
2 grade = student_grades["Charlie"] # Raises KeyError
```

- ♦ *ValueError*: A function receives an argument of the correct type but an inappropriate value.

```
1 number_str = "abc"
2 integer_value = int(number_str) # Raises ValueError
```

- ♦ *FileNotFoundError*: Trying to open a file that does not exist (subclass of *OSError*).

```
1 with open("non_existent_file.txt", "r") as f:
2     content = f.read() # Raises FileNotFoundError
```

- ♦ *AttributeError*: Attempting to access an attribute or method that an object doesn't have.

```
1 my_number = 10
2 my_number.append(5) # Raises AttributeError ('int' object has no attribute 'append')
```


Handling Exceptions: try and except

- ◆ Purpose: To gracefully handle exceptions that might occur in a specific block of code.

- ◆ Syntax:

```
1  try:
2      # Code that might raise an exception
3      # <potentially_problematic_code>
4      print("Attempting operation...")
5      result = 10 / 0 # This will raise ZeroDivisionError
6      print("Operation successful!") # This line won't be reached
7  except <ExceptionType>:
8      # Code to run if the specified exception occurs
9      # <handler_code>
10     print("An error occurred! Cannot divide by zero.")
11
12     print("Program continues after handling.")
```

- ◆ Flow:

- ◆ Code inside the *try* block is executed.
- ◆ If **No** exception occurs, the *except* block is **skipped**.
- ◆ If an exception occurs that **matches** *<ExceptionType>*, the rest of the *try* block is **skipped**, and the *except* block is **executed**.
- ◆ Execution continues after the entire *try...except* block.

Practical Example: Safe Division

```
1  def safe_divide(numerator, denominator):
2      print(f"Attempting to divide {numerator} by {denominator}")
3      try:
4          result = numerator / denominator
5          print(f"Result is: {result}")
6          return result
7      except ZeroDivisionError:
8          print("Error: Cannot divide by zero!")
9          return None # Return a default value or indicator
10
11 # Test cases
12 safe_divide(10, 2)
13 print("-" * 20)
14 safe_divide(10, 0)
15 print("-" * 20)
16 safe_divide(5, -1)
```

```
Attempting to divide 10 by 2
Result is: 5.0
-----
Attempting to divide 10 by 0
Error: Cannot divide by zero!
-----
Attempting to divide 5 by -1
Result is: -5.0
```

Being Precise: Catching Different Errors

- You can have multiple `except` blocks to handle different exception types specifically.
- Python checks the `except` blocks in order. The first one matching the raised exception is executed.

```
1 def process_value(value):
2     try:
3         number = int(value) # Might raise ValueError
4         result = 100 / number # Might raise ZeroDivisionError
5         print(f"Result of 100 / {number} is {result}")
6     except ValueError:
7         print(f"Error: '{value}' is not a valid integer.")
8     except ZeroDivisionError:
9         print("Error: Cannot divide by zero (input was 0).")
10    except Exception as e: # Catch any other unexpected exceptions
11        print(f"An unexpected error occurred: {type(e).__name__}")
12
13 process_value("5")
14 process_value("abc")
15 process_value("0")
16 process_value([]) # Example of unexpected error
```

```
Result of 100 / 5 is 20.0
Error: 'abc' is not a valid integer.
Error: Cannot divide by zero (input was 0).
An unexpected error occurred: TypeError
```

Grouping Handlers

- ◆ If the handling logic is the same for multiple exception types, you can **group** them in a **tuple**.
- ◆ Syntax:

```
1  try:
2      # Code that might raise IndexError or KeyError
3      data = {'a': [1, 2]}
4      key = 'b'
5      index = 0
6      value = data[key][index] # Could raise KeyError or IndexError
7      print(f"Value found: {value}")
8  except (KeyError, IndexError):
9      # Handle both KeyError and IndexError here
10     print("Error: Invalid key or index used to access data.")
11
12     print("Continuing execution...")
```

Getting More Information: The as Keyword

- ◆ You can get access to the exception object itself using `as <variable_name>`.
- ◆ This **object** often contains useful attributes, like the **error message**.

```
1  try:
2      x = int("not a number")
3  except ValueError as err:
4      print(f"A ValueError occurred!")
5      print(f"Error type: {type(err)}")
6      print(f"Error message: {err}") # Printing the object often gives the message
7
8  print("Program finished handling the error.")
```

```
A ValueError occurred!
Error type: <class 'ValueError'>
Error message: invalid literal for int() with base 10: 'not a number'
Program finished handling the error.
```

Code That Runs Only on Success: else

- ◆ Purpose: To execute code **only** if the try block completes **without** raising any exceptions.
- ◆ Benefit: Separates the "success" code from the main try block, making the *try* block shorter and focused on the potentially problematic operations.
- ◆ Syntax:

```
1  try:
2      # Code that might raise an exception
3      user_input = input("Enter a number: ")
4      number = int(user_input) # Potential ValueError
5  except ValueError:
6      print("Invalid input. Please enter a valid number.")
7  else:
8      # This runs ONLY if NO exception occurred in the try block
9      print(f"Successfully converted! The number is {number}")
10     # Further processing using 'number' can go here
```

Guaranteed Execution: finally

- ◆ Purpose: To execute **cleanup code** regardless of whether an exception occurred or not.
- ◆ Use Cases: Closing files, releasing network connections, database cleanup, releasing locks.

```
1 file = None # Initialize to None
2 try:
3     print("Attempting to open file...")
4     file = open("my_data.txt", "w") # Potential IOError/FileNotFoundError etc.
5     print("File opened. Writing data...")
6     file.write("Important data!")
7     # Simulate an error during processing (e.g., uncomment below)
8     # result = 10 / 0
9     print("Data written successfully.")
10 except IOError as e:
11     print(f"An error occurred opening or writing to the file: {e}")
12 except ZeroDivisionError:
13     print("Calculation error occurred during processing.")
14 finally:
15     # This code ALWAYS runs, whether an exception occurred or not.
16     print("Entering finally block...")
17     if file: # Check if file was successfully opened
18         print("Closing file.")
19         file.close()
20     else:
21         print("File was not opened.")
22     print("Exiting finally block.")
23
24 print("Program continues after try/except/finally.")
```


Putting It All Together

- ♦ Execute try block.
- ♦ If Exception:
 - ♦ Find matching except block.
 - ♦ If found: Execute except block.
 - ♦ If not found: Exception propagates upwards.
 - ♦ Execute finally block (if present).
- ♦ If No Exception:
 - ♦ Execute else block (if present).
 - ♦ Execute finally block (if present).
- ♦ Continue execution after the entire structure (unless an unhandled exception occurred).

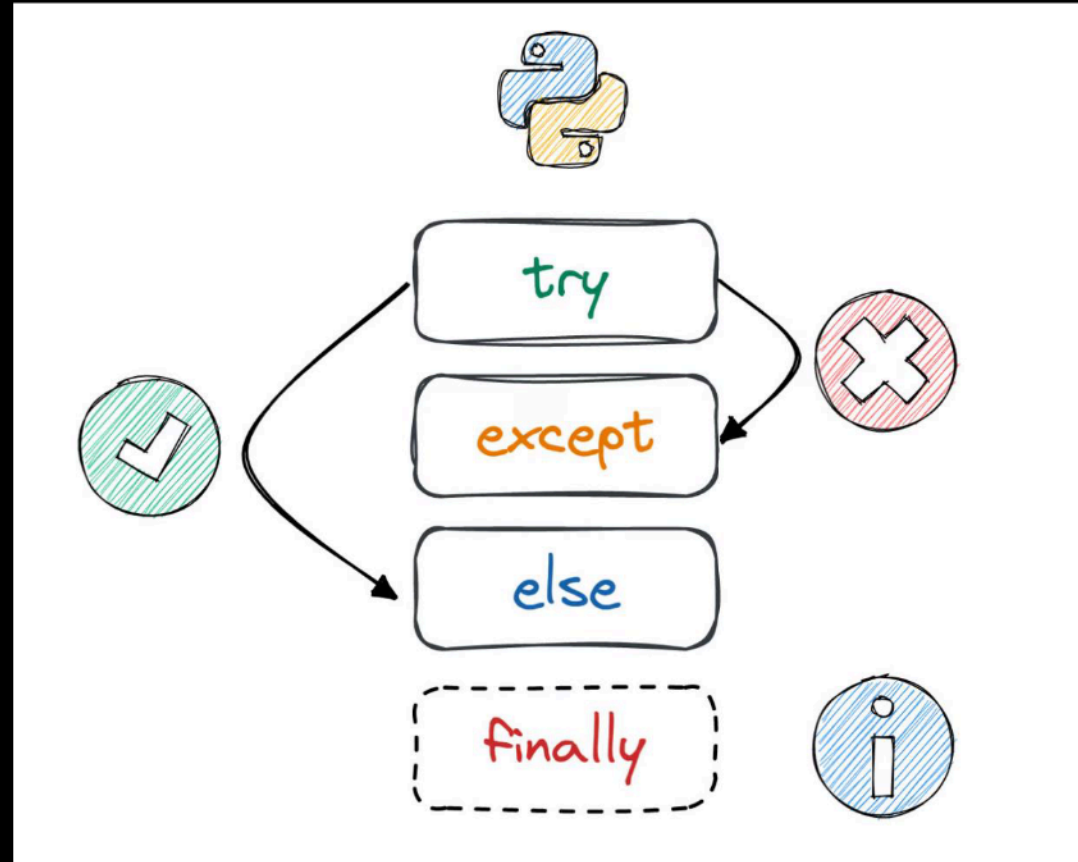


Image from Abid Ali Awan, Dec 12, 2024, <https://www.datacamp.com/tutorial/exception-handling-python>

Test Your Understanding

- What will be printed by the following code snippets?

```
1  try:
2      print("Start Try")
3      # No exception
4      print("End Try")
5  except:
6      print("Caught Exception")
7  else:
8      print("Else Block")
9  finally:
10     print("Finally Block")
```

```
1  try:
2      print("Start Try")
3      x = 1 / 0 # Exception!
4      print("End Try")
5  except ZeroDivisionError:
6      print("Caught Exception")
7  else:
8      print("Else Block")
9  finally:
10     print("Finally Block")
```

```
1  try:
2      print("Start Try")
3      x = 1 / 0 # Exception!
4      print("End Try")
5  except TypeError: # Doesn't match
6      print("Caught Exception")
7  else:
8      print("Else Block")
9  finally:
10     print("Finally Block") # Runs even if unhandled
```

Test Your Understanding

- What will be printed by the following code snippets?

```
1 try:
2     print("Start Try")
3     # No exception
4     print("End Try")
5 except:
6     print("Caught Exception")
7 else:
8     print("Else Block")
9 finally:
10    print("Finally Block")
```

```
Start Try
End Try
Else Block
Finally Block
```

```
1 try:
2     print("Start Try")
3     x = 1 / 0 # Exception!
4     print("End Try")
5 except ZeroDivisionError:
6     print("Caught Exception")
7 else:
8     print("Else Block")
9 finally:
10    print("Finally Block")
```

```
Start Try
Caught Exception
Finally Block
```

```
1 try:
2     print("Start Try")
3     x = 1 / 0 # Exception!
4     print("End Try")
5 except TypeError: # Doesn't match
6     print("Caught Exception")
7 else:
8     print("Else Block")
9 finally:
10    print("Finally Block") # Runs even if unhandled
```

```
Start Try
Finally Block
Traceback (most recent call last):
  File "/Users/chenhaoxu/Downloads/test.py", line 3, in <module>
    x = 1 / 0 # Exception!
ZeroDivisionError: division by zero
```

Defining Your Own Error Types

Why?

- ◆ Built-in exceptions might be **too general** (*ValueError*, *RuntimeError*).
- ◆ To represent **specific error conditions** unique to your application's domain (e.g., *InsufficientFundsError*, *InvalidUserInputError*, *NetworkTimeoutError*).
- ◆ Makes code more **readable** and **understandable** – catching specific custom exceptions tells you exactly what went wrong in your application's context.
- ◆ Allows **grouping** related application errors under a common base class.

How?

- ◆ Create a new **class** that **inherits** (directly or indirectly) from the built-in *Exception* class.
- ◆ You can add attributes and customize the `__init__` method to store more context about the error.

Custom Exception - Example

```
1 class TemperatureError(Exception):
2     """Base class for exceptions in our temperature processing system."""
3     pass # We can add more later, like custom attributes or messages
4
5
6 class BelowAbsoluteZeroError(TemperatureError):
7     """Raised when the temperature is below absolute zero."""
8     def __init__(self, celsius):
9         self.celsius = celsius
10        message = f"Temperature {celsius}°C is below absolute zero."
11        super().__init__(message)
12
13 def celsius_to_fahrenheit(celsius):
14     if celsius < -273.15:
15         raise BelowAbsoluteZeroError(celsius)
16     return (celsius * 9/5) + 32
17
18 try:
19     temp_c = -280.0
20     temp_f = celsius_to_fahrenheit(temp_c)
21     print(f"Temperature in Fahrenheit: {temp_f}")
22 except BelowAbsoluteZeroError as e:
23     print(f"Error: {e}") # More descriptive message
```

Error: Temperature -280.0°C is below absolute zero.

Custom Exception - Example

```
1 class InsufficientFundsError(Exception):
2     """Exception raised when a withdrawal exceeds available balance."""
3     def __init__(self, balance, amount_needed, message="Insufficient funds for transaction."):
4         self.balance = balance
5         self.amount_needed = amount_needed
6         self.message = f"{message} Available: ${balance:.2f}, Needed: ${amount_needed:.2f}"
7         # Call the base class __init__ with the message
8         super().__init__(self.message)
9
10    def withdraw(current_balance, withdraw_amount):
11        if withdraw_amount > current_balance:
12            raise InsufficientFundsError(current_balance, withdraw_amount)
13        else:
14            print(f"Withdrawing ${withdraw_amount:.2f}. New balance: ${current_balance - withdraw_amount:.2f}")
15            return current_balance - withdraw_amount
16
17    # Example Usage
18    account_balance = 100.00
19    try:
20        account_balance = withdraw(account_balance, 50.00)
21        account_balance = withdraw(account_balance, 75.00) # Raises InsufficientFundsError
22    except InsufficientFundsError as e:
23        print(f"Transaction Failed: {e}")
24        # We can access the custom attributes
25        print(f"Details - Current Balance: ${e.balance:.2f}, Attempted Withdrawal: ${e.amount_needed:.2f}")
```

Withdrawing \$50.00. New balance: \$50.00

Transaction Failed: Insufficient funds for transaction. Available: \$50.00, Needed: \$75.00

Details - Current Balance: \$50.00, Attempted Withdrawal: \$75.00

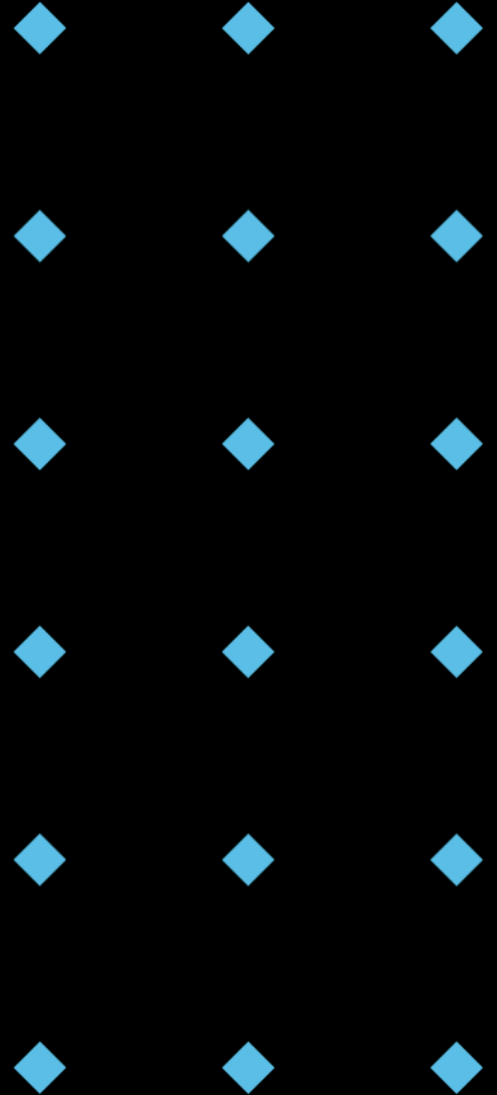
Intentionally Triggering Exceptions: raise

Purpose:

- ♦ To signal that an error condition (detected by your own logic) has occurred. Used often with **custom exceptions** or to **enforce conditions**.
- ♦ To **re-raise** a caught exception, allowing it to be handled by an outer *try...except* block (propagation).

Syntax:

- ♦ *raise* <ExceptionType>(<optional_arguments>) - Raise a new exception.
- ♦ *raise* (used inside an except block) - Re-raise the currently handled exception.



Example: raise for Validation

```
1 def set_temperature(celsius):
2     # Absolute zero in Celsius is -273.15
3     if celsius < -273.15:
4         # Raise a built-in exception for invalid value
5         raise ValueError("Temperature cannot be below absolute zero (-273.15 C).")
6     elif celsius > 3000: # Arbitrary upper limit for example
7         # Can use custom exceptions too
8         # class TemperatureTooHighError(Exception): pass
9         # raise TemperatureTooHighError("Temperature is unrealistically high.")
10        raise ValueError("Temperature seems unrealistically high.")
11    else:
12        print(f"Temperature set to {celsius}°C.")
13
14    try:
15        set_temperature(25)
16        set_temperature(-300) # Raises ValueError
17    except ValueError as e:
18        print(f"Error setting temperature: {e}")
19
20    try:
21        set_temperature(5000) # Raises ValueError
22    except ValueError as e:
23        print(f"Error setting temperature: {e}")
```

Temperature set to 25°C.

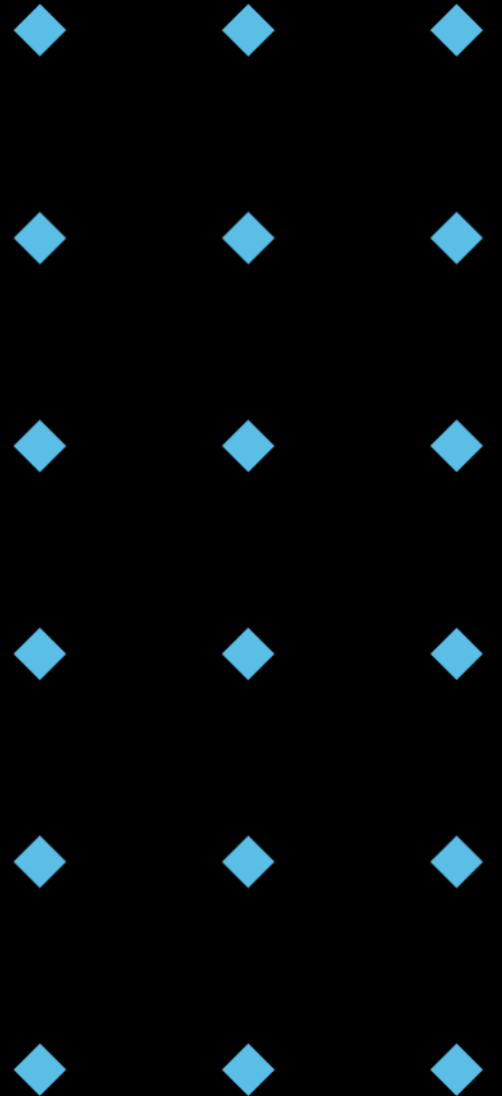
Error setting temperature: Temperature cannot be below absolute zero (-273.15 C).

Error setting temperature: Temperature seems unrealistically high.

Example: Logging and Re-raising

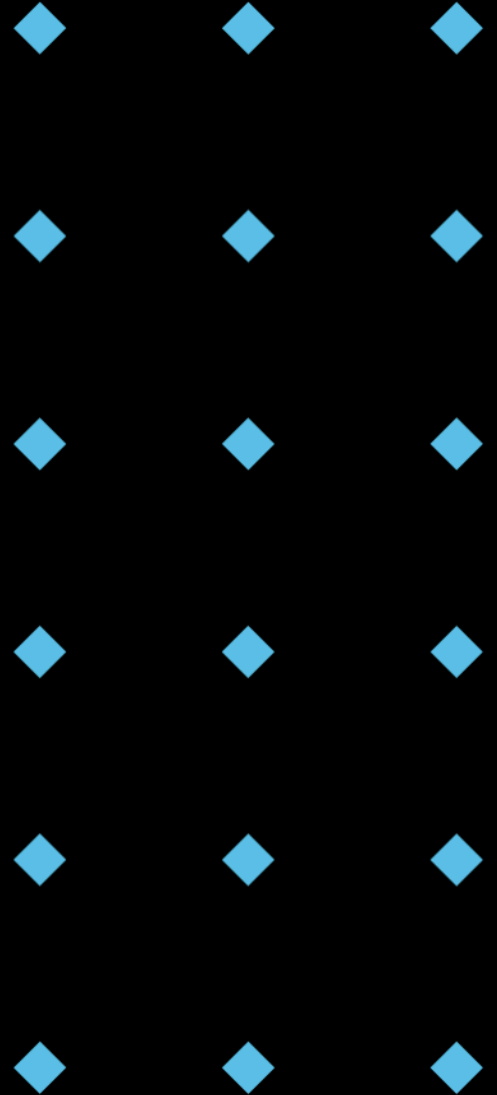
```
1 import logging
2
3 logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')
4
5 def potentially_failing_operation(data):
6     try:
7         # Simulate an operation that might fail
8         key = 'required_key'
9         value = data[key]
10        print(f"Successfully accessed key '{key}', value: {value}")
11        return value
12    except KeyError as e:
13        # Log the error first
14        logging.error(f"Missing key in data: {e}")
15        # Then re-raise the exception for outer handlers
16        raise
17
18 # Outer handling layer
19 my_data = {'other_key': 100}
20 try:
21     result = potentially_failing_operation(my_data)
22     print("Operation completed successfully.")
23 except KeyError:
24     print("Handling the KeyError at the higher level: Data structure was invalid.")
25     # Maybe provide default value or exit gracefully
```

ERROR: Missing key in data: 'required_key'
Handling the KeyError at the higher level: Data structure was invalid.



Custom vs. Built-in

- ◆ Scenario: You are building a system for processing online orders. What are some error conditions where a **custom** exception might be better than a **built-in** one?
- ◆ Examples to consider:
 - ◆ Product out of stock
 - ◆ Invalid coupon code
 - ◆ Payment declined
 - ◆ Shipping address invalid
- ◆ Discussion: For each, would a built-in exception (*ValueError*, *KeyError*, *RuntimeError* etc.) be sufficient, or would a custom exception (*OutOfStockError*, *InvalidCouponError*, *PaymentFailedError*) be clearer? Why?



How Exceptions Bubble Up

- When an exception is raised, if the current function doesn't have a matching except block, Python looks for a handler in the **calling function**.
- This continues up the **call stack** (the chain of function calls that led to the current point).
- If the exception reaches the top level of the program without being caught, the program **terminates** and prints a **traceback**.

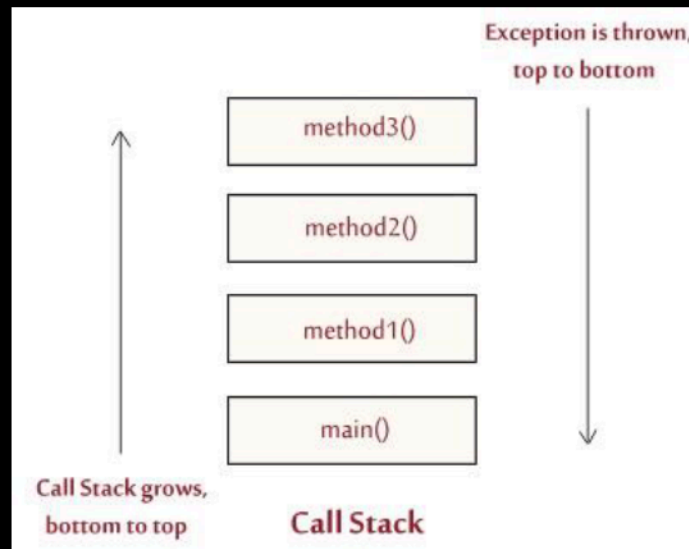


Image from Carpentries, July 6 2025, <https://carpentries-incubator.github.io/pando-python/instructor/profiling-functions.html>

```
1 def function_c():
2     print("Entering C")
3     value = 1 / 0 # Raises ZeroDivisionError here
4     print("Leaving C") # Not reached
5
6 def function_b():
7     print("Entering B")
8     function_c() # Exception propagates from C to here
9     print("Leaving B") # Not reached
10
11 def function_a():
12     print("Entering A")
13     try:
14         function_b() # Exception propagates from B to here
15     except ZeroDivisionError:
16         print("Caught the division error in function A!")
17         print("Leaving A")
18
19 # Main execution
20 print("Starting program")
21 function_a()
22 print("Program finished")
```

```
Starting program
Entering A
Entering B
Entering C
Caught the division error in function A!
Leaving A
Program finished
```

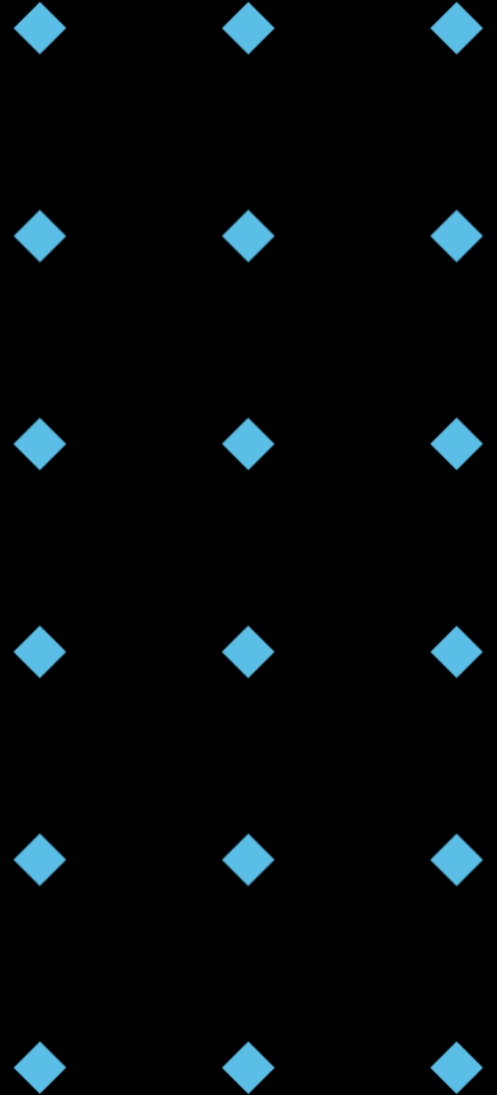
To Catch or Not To Catch?

When should a function **catch** an exception, and when should it let it **propagate** upwards?

Consider:

- ◆ Can the current function **meaningfully** handle the error? (e.g., retry, use a default value, log it specifically?)
- ◆ Does the current function have **enough context** to handle it appropriately?
- ◆ Is the error specific to **this function's internal logic**, or is it a more general failure the caller should know about?
- ◆ Would handling it here **hide a critical problem** from higher levels?

General Guideline: Handle an exception where you can actually do something meaningful about it. Otherwise, let it propagate (or catch, log, and re-raise).



Best Practice 1: Be Specific

- ◆ Catch Only What You Expect
- ◆ **Avoid:** Catching **overly broad exceptions** like `Exception` or `BaseException` unless you have a very specific reason (e.g., top-level logging).
- ◆ **Prefer:** Catching **specific exceptions** you anticipate and know how to handle.
- ◆ **Why?** Prevents accidentally masking unrelated bugs and makes intent clearer.

```
1 # Less Recommended (can hide unexpected errors like TypeError, NameError)
2 try:
3     user_input = input("Enter number: ")
4     result = 10 / int(user_input)
5 except Exception as e: # Catches ValueError, ZeroDivisionError, and potentially others!
6     print(f"An error occurred: {e}")
```

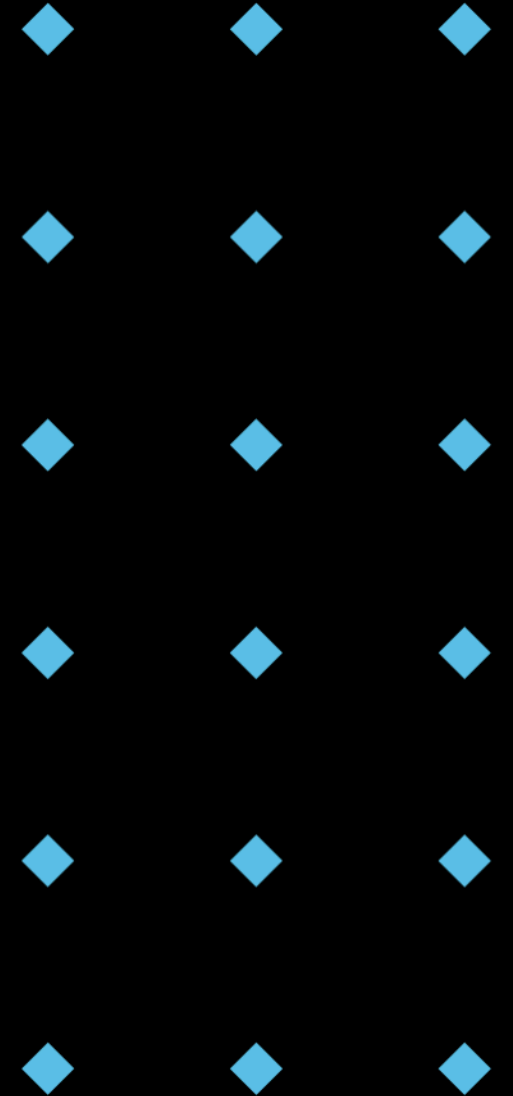
```
1 # Recommended
2 try:
3     user_input = input("Enter number: ")
4     result = 10 / int(user_input)
5 except ValueError:
6     print("Invalid input: Please enter a number.")
7 except ZeroDivisionError:
8     print("Cannot divide by zero.")
9 # Let other unexpected exceptions (like KeyboardInterrupt) propagate
```

Best Practice 2: Meaningful Messages & Custom Exceptions

- ◆ When raising exceptions (especially custom ones), provide **clear, informative** messages.
- ◆ Include **relevant context** if possible.
- ◆ Use custom exceptions (as discussed) to represent **domain-specific** errors clearly.

```
1 # Less helpful
2 raise ValueError("Input error")
3
4 # More helpful
5 raise ValueError(f"Invalid quantity specified: {qty}. Must be a positive integer.")
```

```
1 # Catching this...
2 except ValueError:
3     # What kind of value error was it?
4
5 # Is less clear than catching this...
6 except OutOfStockError as e:
7     print(f"Sorry, item {e.item_id} is out of stock.")
```



Best Practice 3: Clean Up Resources (finally / with)

- ◆ Use *finally* to ensure essential cleanup code runs (closing files, releasing locks, closing network connections).
- ◆ Even Better: Use the *with* statement (**Context Manager**) for resources that support it. It **automatically** handles cleanup, even if exceptions occur.
- ◆ The *with* statement implicitly uses concepts related to *finally*.

```
1  # Using finally (Correct but verbose)
2  f = None
3  try:
4      f = open("data.txt", "r")
5      # process file content...
6      # if something_bad_happens: raise Exception()
7  finally:
8      if f:
9          f.close()
10
11 # Using 'with' (Preferred, cleaner, safer)
12 try:
13     with open("data.txt", "r") as f: # Automatically calls f.close() at the end
14         # process file content...
15         # if something_bad_happens: raise Exception()
16 except FileNotFoundError:
17     print("File not found.")
18 # No need for finally to close 'f', 'with' handles it!
```

Best Practice 4: Keep Handlers Simple

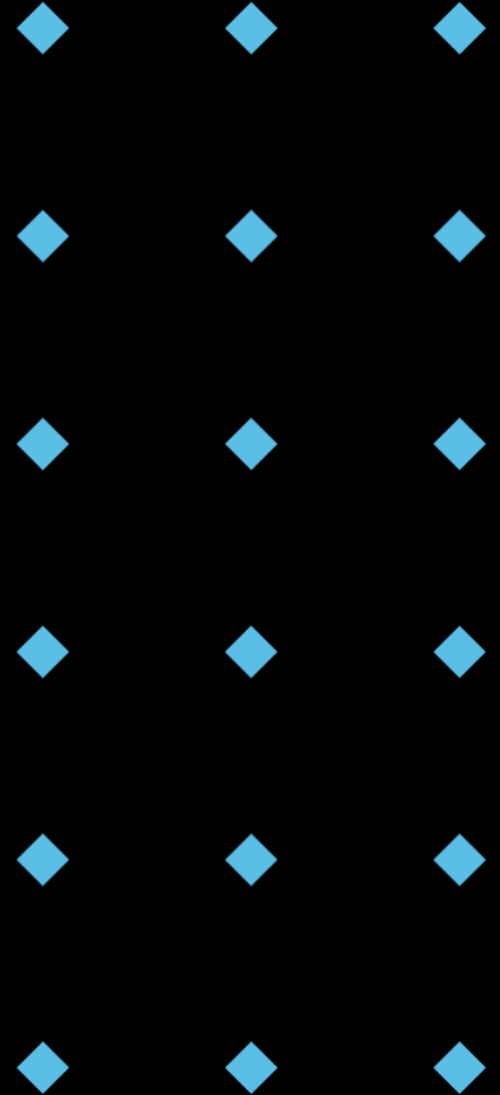
Keep try and except Blocks Focused

- ◆ **Minimize code inside try blocks:** Only include the specific lines that might raise the exception(s) you are handling. This makes it clearer what errors are being caught.
- ◆ **Keep except blocks simple:** Handlers should focus on dealing with the exception (logging, cleanup, returning defaults). Avoid complex application logic inside except blocks. If recovery is complex, call a separate function.

```
1  # Less Clear: Too much in the try block
2  try:
3      # Line 1: Might raise ErrorA
4      # Line 2: OK
5      # Line 3: OK
6      # Line 4: Might raise ErrorB
7      # Line 5: OK
8  except (ErrorA, ErrorB):
9      # Which one failed? Harder to tell.
10
11 # Better: Focused try blocks
12 try:
13     # Line 1: Might raise ErrorA
14 except ErrorA:
15     # Handle ErrorA
16 else: # If Line 1 succeeded
17     # Line 2, 3
18     try:
19         # Line 4: Might raise ErrorB
20     except ErrorB:
21         # Handle ErrorB
22 else: # If Line 4 succeeded
23     # Line 5
```


Key Takeaways

- ◆ Exceptions are Normal: Runtime errors are a fact of life in programming.
- ◆ Handling is Key: try, except, else, finally provide structured control flow for errors.
- ◆ Context Matters: Custom exceptions, raise, propagation, and chaining (raise from) help manage errors effectively in complex applications.
- ◆ Best Practices: Following guidelines leads to robust, maintainable, and readable code.



Q & A



Lab

